

Rat26F Lexical Analyzer

By Simone Bacani, Vibhor Bhargava, Aaron Yu

CS323 Documentation

About 2 pages

1. Problem Statement

We need to write a lexical analyzer for the Rat26F programming language so Rat26F source code can be parsed into tokens.

2. How to use your program

If you haven't already, install a Python interpreter, then run:

```
python assignment1.py <input file> [output file]
```

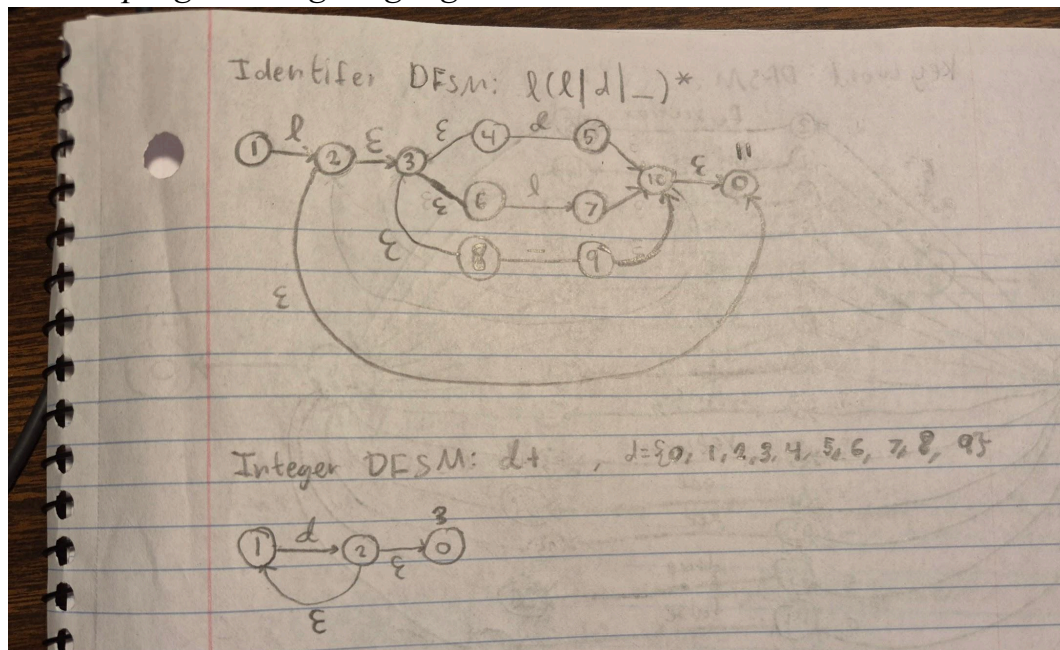
- <input file>: the Rat26F source file to analyze.
- [output file]: optional. If omitted, results are written to output.txt.

Example:

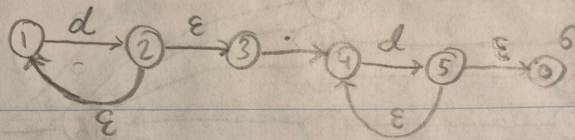
```
python assignment1.py input1.txt output1.txt
```

3. Design of your program

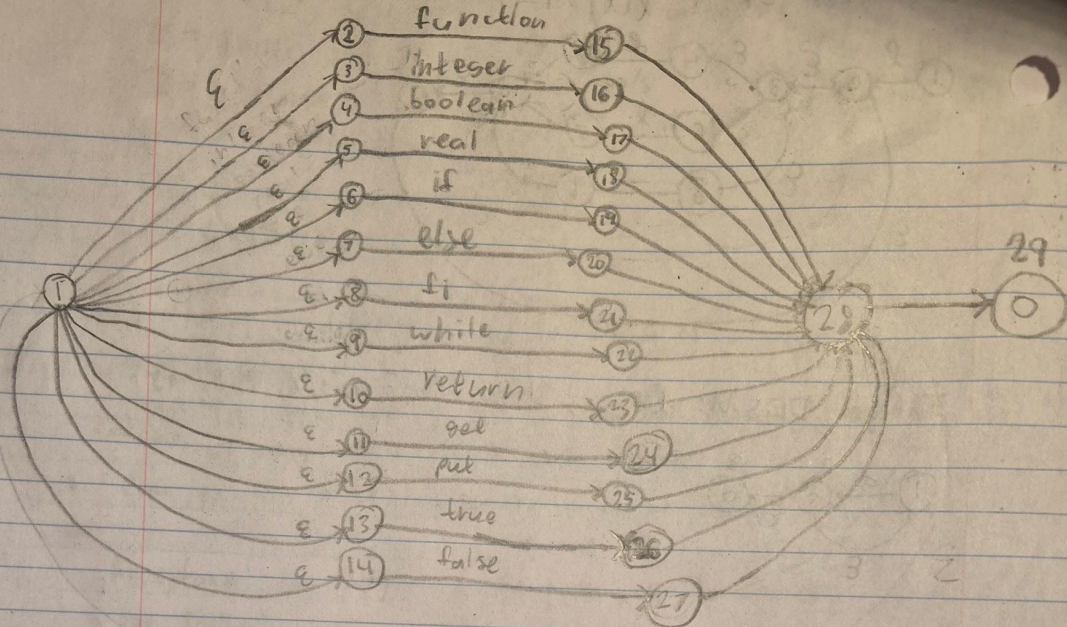
We designed our program by using the following deterministic finite state machines (DFSMs). Each DFSM represents a token type in the Rat26F programming language.



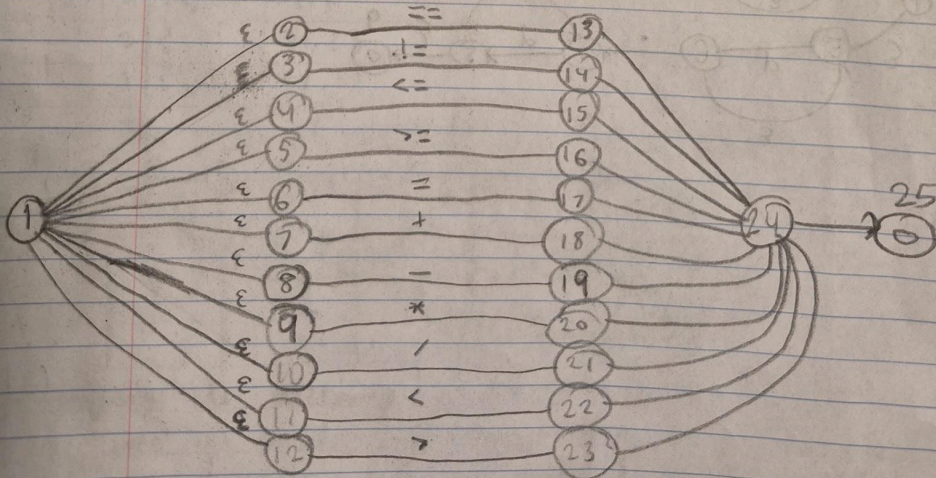
real DFSA: $(d^*) \cdot (d^+)$



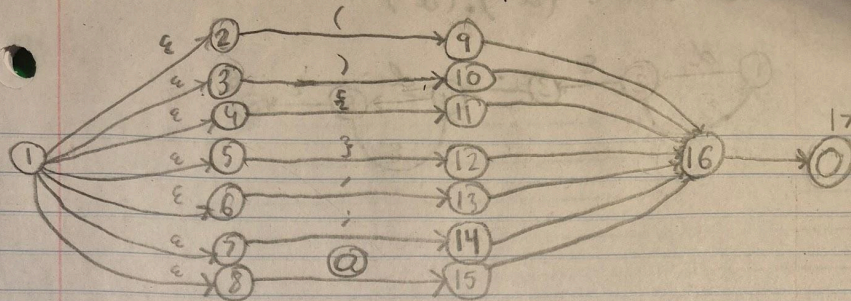
keyword DFSA



Operator DFSA



separator DFSM:



unknown DFSM:

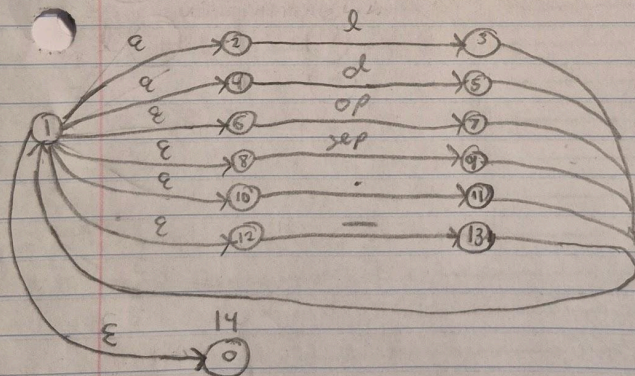
*unknown = any character that's not part of Rule 26F

Let l = every letter in English alphabet

d = all base-10 digits

op = operators

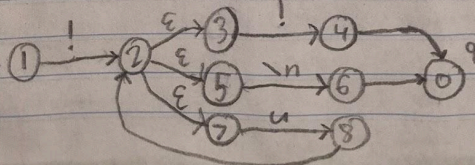
sep = separators



error DFSM:

*error = unterminated comment

Let u = every unicode character



We then translated these DFSMs into Python code by representing them as Finite State Machine State Transition Tables. To assist with this, we used Python dictionaries to define character sets (e.g. LETTERS for letters of the

English alphabet and DIGITS for digits of base-10), and wrote a function `run_dfsm` which iterates through a DFMSM passed to the function. Additionally, whitespace characters are skipped and comments are ignored.

4. Any Limitation

*<All features are running according to the assignment but you limit your program due to resource limitations, such as Maximum number of lines in the source code, size of the identifier, integer etc. **Say 'None' if there is no limitation**>*

None

5. Any shortcomings

*<Anything you could NOT implement although that is required by the Assignment. **Say 'None' if there is no shortcoming**>*

None